

Pointers

Whenever we create a variable, the computer stores that data in a specific location in its **memory (RAM)**. A **pointer** is exactly what it sounds like: it is a variable that points to the memory address of another variable, rather than holding the actual data itself.

1. The Two Crucial Operators (& and *)

- **The Ampersand (&) - "Address-of" Operator:** If you put an **&** in front of an existing variable, it returns the physical memory address of that variable.
- **The Star (*) - "Dereference" Operator:** If you put a ***** in front of a memory address (**a pointer**), it travels to that address and returns the actual underlying data stored there.

2. Step-by-Step Example

Let's break down and see how **&** and ***** work together:

```
var x int = 1
var y int

var ip *int

ip = &x
y = *ip // 1
```

1. We create an integer **x** and set it to **1**. This "**1**" is now sitting somewhere in memory.
2. We create an integer **y**. Since we didn't give it a value, **Go defaults it to 0**.
3. We create a pointer named **ip**. The ***int** type means "this will hold the address of an integer").
4. We use **&** to find the address of **x**, and we save that address inside **ip**.
5. We use ***** to look inside the address stored in **ip**. It finds the number 1 and assigns that value to **y**.

By the end of this sequence, **y** is safely equal to **1**.

3. The new() Function

Go gives us a built-in function called **new()** as a shortcut for working with pointers.

Normally, when we declare a variable using **var**, Go gives us the variable itself. If we use **new()**, Go automatically creates a hidden, nameless variable, initializes it to its "**Zero Value**", and immediately hands you back the **pointer**

Example:

```
ptr := new(int)
*ptr = 3
```

1. Go allocates memory for an integer, sets it to **0**, and ptr now holds that address.
2. We use the ***** operator to travel to that address and overwrite the **0** with a **3**.

4. Expert Takeaway:

When passing large data structures (like massive **JSON**) between different functions in your backend, passing them by their normal value means the **server has to create a heavy duplicate of that data every single time**.

By passing a **pointer** instead, you are **just handing over a tiny memory address**. This is one of the key secrets to Go's blazing-fast performance and incredibly low memory footprint when building scalable web applications.

To truly understand the advantage of **pointers**, refer to the **OOPs** concept

Pass by value vs. Pass by reference